지능형로보틱스 최종발표

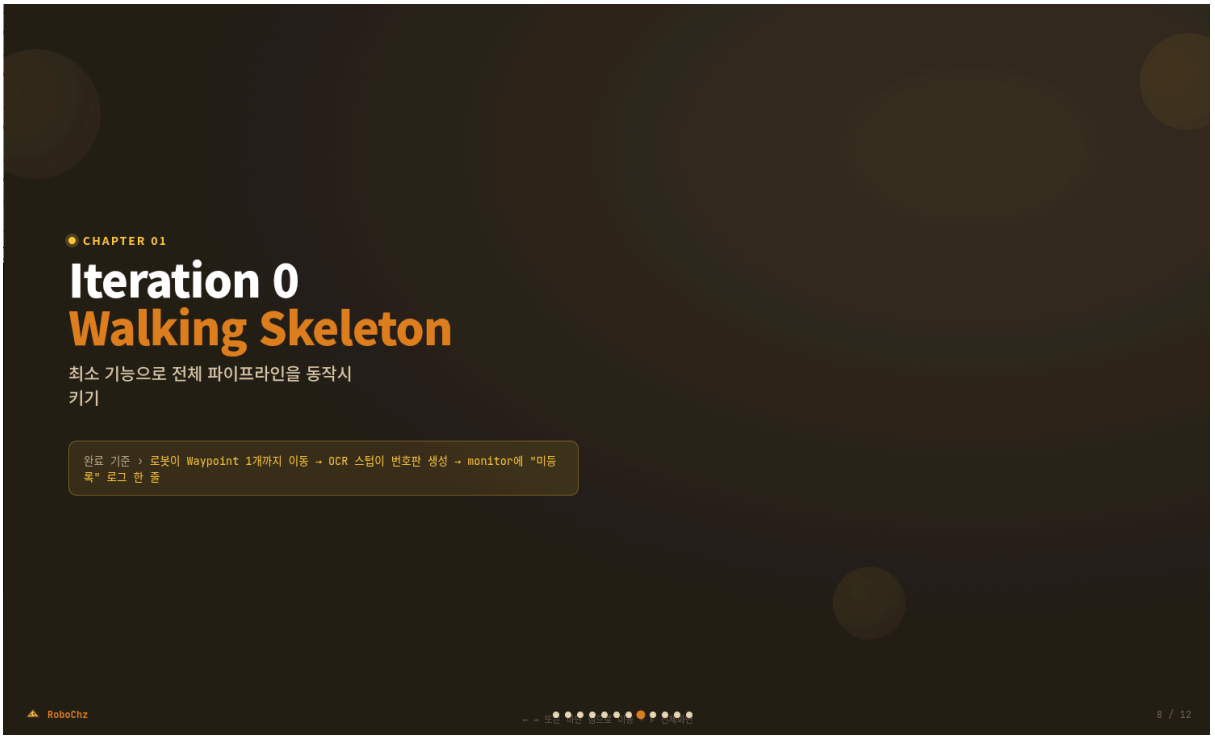
2. 프로젝트 정의

아파트 불법 주차 문제

- 아파트 내 주차 공간 부족 문제로 인해 불법 주차 단속은 필수적이다. 일반적으로는 경비원 또는 관리인이 심야에 직접 순찰하며 위반 차량에 스티커를 부착하는 방식이 사용된다. 그러나 이 방식은 심야 인력 운용에 따른 비용과 번거로움이 발생할 뿐 아니라, 스티커 부착에 반발한 입주민이 접착제 제거 비용을 청구하거나 흉기로 위협하는 등 민형사상 분쟁으로 번지는 사례가 잇따르고 있다. 법적으로도 접착식 스티커를 재물손괴로 판단한 사례가 존재해 관리 주체의 법적 부담이 크다.
- 차단봉 및 차단기 방식은 입구마다 설치해야 하므로 비용이 크고, 노후 아파트 단지에 즉시 적용하기 어렵다. 또한 입주민의 일상적인 출입까지 불편하게 만들며, 택배 및 방문 차량에 대한 예외처리가 번거롭다는 단점이 있다.

제안: 자율 순찰 단속 로봇

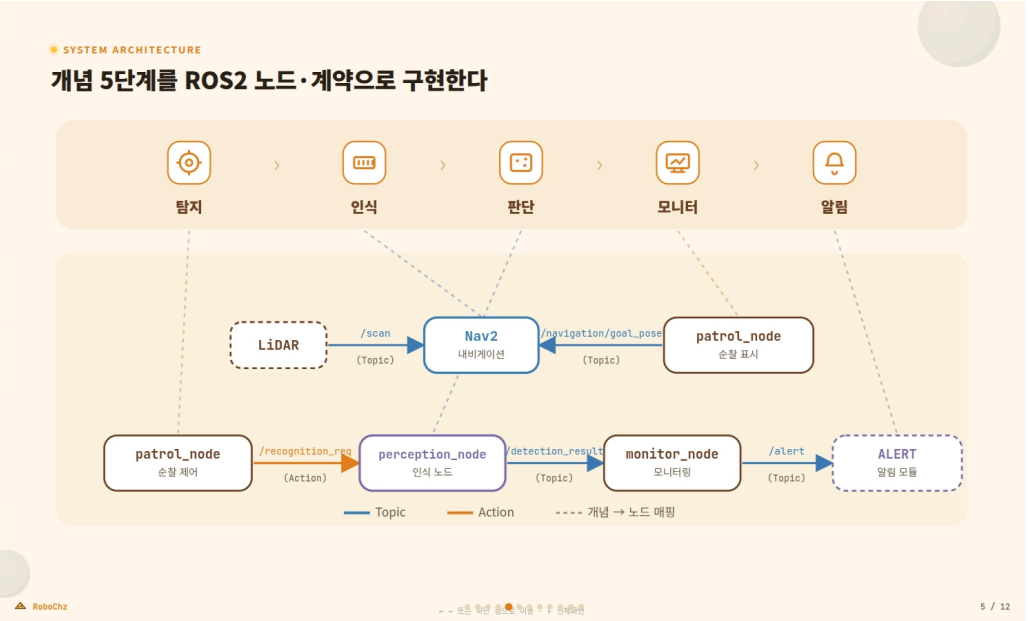
- 위 문제를 해결하기 위해 외부 차량 금지 시간대에 로봇이 주차장을 자율 순찰하며 번호판을 인식하고, 미등록 차량 발견 시 번호판 이미지와 주차 위치를 기록하여 관리자에게 알림을 전송하는 시스템을 제안한다.
- 이러한 방식은 스티커 부착 없이 디지털 증거 기반으로 단속하므로 입주민과의 물리적 충돌 및 법적 분쟁 가능성을 줄일 수 있다. 또한 차단기 방식 대비 설치 비용이 낮고, 기존 아파트 단지 구조에 상관없이 보편적으로 적용할 수 있다는 장점이 있다.



STEP1) ROS2 패키지 생성 및 인터페이스 설계

- package 생성
  1. robochz_msgs : 메시지/액션 정의
  2. robochz_patrol : Nav2 액션 호출
  3. robochz_perception : 카메라/OCR/라이다
  4. robochz_monitor: 로그/알림/기록

- 인터페이스 설계




```
LiDAR ──(Topic: /scan)──> Nav2

robochz_patrol ──(Action)──> Nav2
  nav2_msgs/action/NavigateThroughPoses

robochz_patrol ──(Action)──> robochz_perception
    robochz_msgs/action/RecognizePlate
  waypoint 도착 후 촬영 및 OCR 요청
  Service도 괜찮지만 YOLO/OCR 수행 시 수초 걸릴 수 있기에 진행 상황 파악을 위해

robochz_perception ──(Topic)──> robochz_monitor
    robochz_msgs/msg/PlateDetection
  번호판 인식 결과 발행

robochz_monitor ──(File/DB)──> patrol_log / report
  순찰 리포트는 ROS 메시지로 별도 발행하지 않고, monitor_node가 PlateDetection 메시지를 누적하여 CSV/JSON 형태로 저장한 뒤 순찰 종료 시 자동 생성한다.
```

- robochz_msgs의 메시지/액션 설계

```
robochz_msgs/
├── action/
│   └── RecognizePlate.action 번호판인식요청 요청/수행
└── msg/
    └── PlateDetection.msg    번호판인식결과 발행/구독
```

```
header:
  stamp: 2026-06-20 15:31:22

waypoint_id: "A-03"

plate_text: "12가3456"
confidence: 0.94
is_registered: false

robot_pose:
  pose:
    position:
      x: 3.42
      y: -1.15
      z: 0.0

image_path: "/logs/images/20260620_153122.jpg"
message: "OCR completed successfully"
```

STEP2) ROS2 패키지 생성 및 인터페이스 설계

- 노드 생성
  - monitor_node (난이도 하)
  - perception_node (난이도 중)
  - patrol_node (난이도 상)
- monitor_node 구현
  - percetion—(Topic)—> monitor
  - 번호판 인식 결과를 받는다.

- PlateDetection Topic Subscriber
- OCR 스텁 Topic을 직접 publish해서 단독 테스트



- perception_node 구현
  - patrol ──(Action)──>perception──(Topic)──> monitor
  - 카메라촬영+OCR수행을 요청받아 번호판인식 결과를 전송한다.
  - Iteration 0에서는 실제 OCR 대신 OCR 스텁 사용
  - 처리흐름

```
1. Goal 수신 (waypoint_id, robot_pose)
2. Feedback 발행
3. 가짜 OCR 생성
4. Topic 발행 (monitor용)
5. Result 반환 (patrol용)
```

  - 동작

```
# 터미널 A: perception 실행
ros2 run robochz_perception perception_node

# 터미널 B: Action Goal 보내기
ros2 action send_goal /recognize_plate robochz_msgs/action/RecognizePlate \
  "{waypoint_id: 'TEST_01', robot_pose: {header: {frame_id: 'map'}}}" --feedback

# 터미널 C에 monitor 실행
ros2 run robochz_monitor monitor_node
```

attachment:9f30c6a4-f95a-4a6c-9a99-657f29151b27:iter0-perception-node2.webm

- patrol_node 구현(1)
  - patrol ──(Action)──>perception
  - 주차칸 이동 완료 후 카메라촬영+OCR수행을 요청한다.
  - 동작

```
# 터미널 A: perception 실행
ros2 run robochz_perception perception_node

# 터미널 B: patrol 실행
ros2 run robochz_patrol patrol_node

# 터미널 C에 monitor 실행
ros2 run robochz_monitor monitor_node
```



STEP3) Nav2 연동하기

- patrol_node 구현(2)
  - patrol ──(Action)──>Nav2
  - patrol노드는 Nav2에게 주차칸 이동 수행을 요청한다.
  - ROS2 기본 제공 TurtleBot3 월드를 이용해 동작검증

```
# 터미널 1 : Gazebo 시뮬레이션 실행
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py

# 터미널 2 : Nav2 실행
ros2 launch turtlebot3_navigation2 navigation2.launch.py \
  use_sim_time:=True \
  map:=$(ros2 pkg prefix turtlebot3_navigation2)/share/turtlebot3_navigation2/map/map.yaml

# 터미널 3 : Monitor
ros2 run robochz_monitor monitor_node

# 터미널 4 : Perception
ros2 run robochz_perception perception_node

# 터미널 5 : Patrol
ros2 run robochz_patrol patrol_node
```

attachment:ba0d279c-20fe-4eb8-8ec2-27ccf92f1b42:iter0-nav2.webm

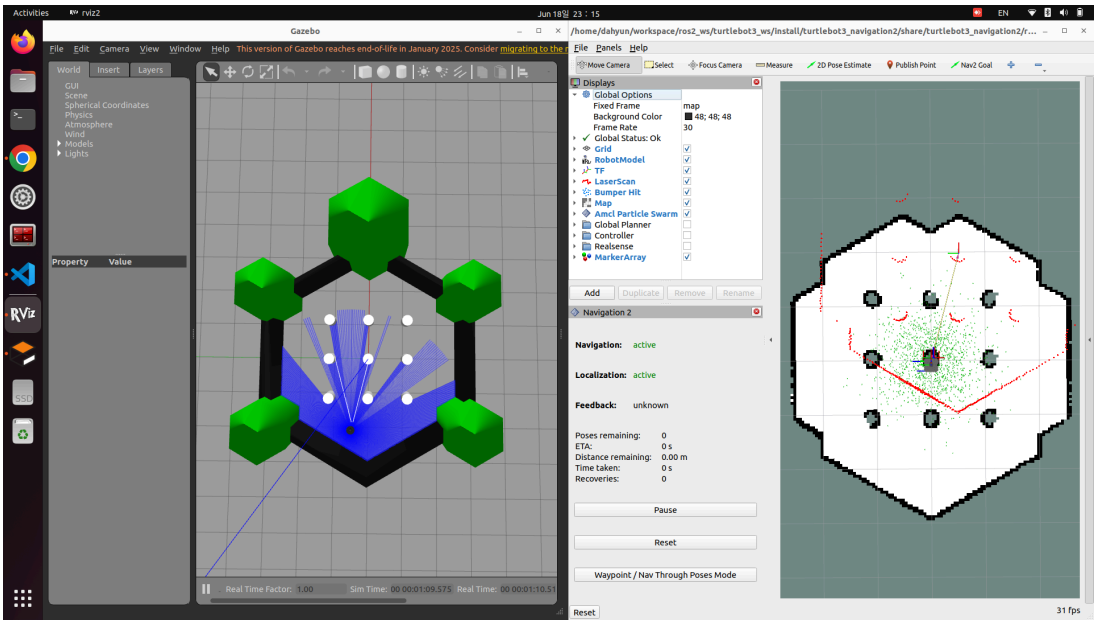
- Iteration 0에서 가장 오래 걸린 트러블 슈팅 (Gazeo 맵위치 ≠ Rviz 맵위치)

```
Gazebo 실제 world
↓
로봇 LiDAR /scan
↓
LaserScan 빨간점은 Gazebo 벽 위치와 잘 맞음

그런데 RViz의 /map 벽 위치가 Gazebo world와 다르게 표시됨
↓
Nav2 costmap이 잘못된 벽/장애물을 기준으로 계산
↓
실제로는 안전한 위치인데 충돌 영역처럼 판단
```

```
↓
Goal 실패
```

LaserScan 빨간점은 Gazebo 벽 위치와 정확한데, RViz의 `/map` 벽 위치가 다르게 표시됨"** 을 짚어준 게 결정적. 두 map 파일을 md5로 비교해서 다른 SLAM 결과임을 확인.



사용 중인 map.yaml이 현재 Gazebo world와 맞지 않음 → 맵 변경

turtlebot3_navigation2/map/map.yaml`은 turtlebot3_world의 SLAM 결과가 **아님** (md5 비교로 확인). 다른 환경의 SLAM | **`nav2_bringup/maps/turtlebot3_world.yaml`이 정확한 짝

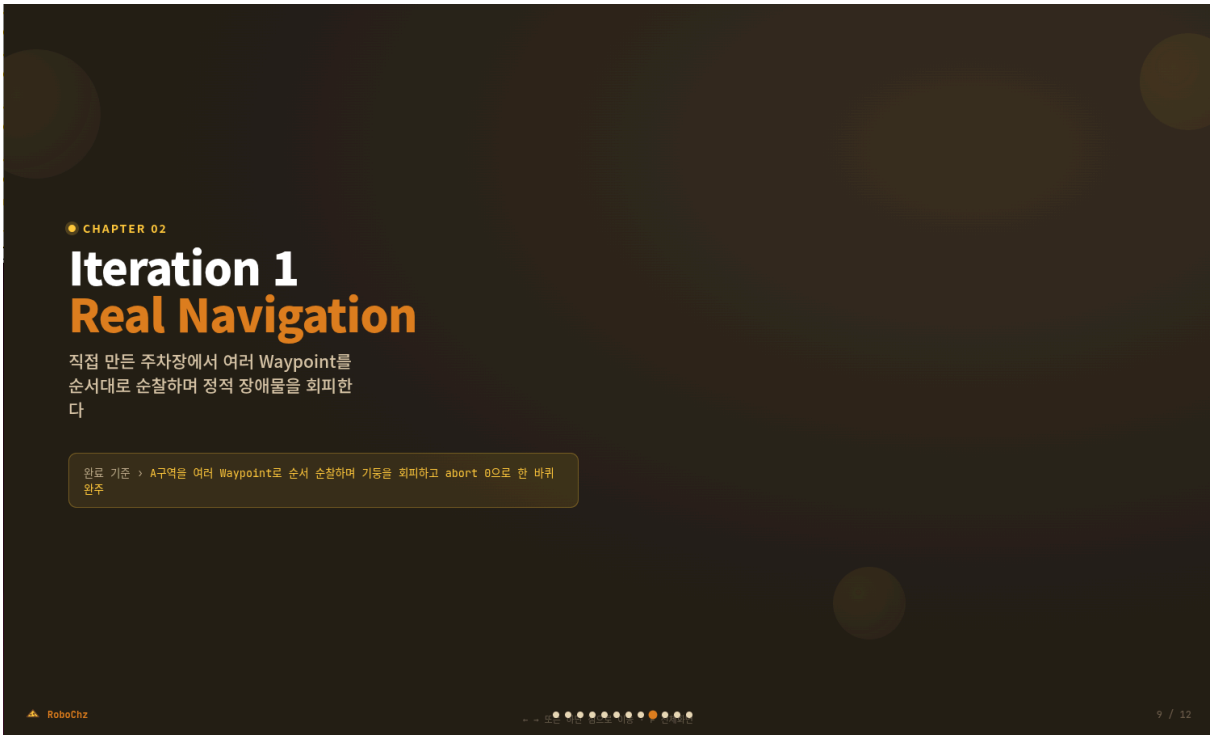
- 최종 파이프라인 동작 검증
  - 두 가지 런치파일로 기능 분류해서 통합
    - sim.launch.py : 시뮬레이션 환경 실행용 런치파일
    - app.launch.py : 순찰·인식·모니터링 노드 실행용 런치파일

```
# 터미널 1: sim
ros2 launch robochz_bringup sim.launch.py

# 터미널 2: app
ros2 launch robochz_bringup app.launch.py
```

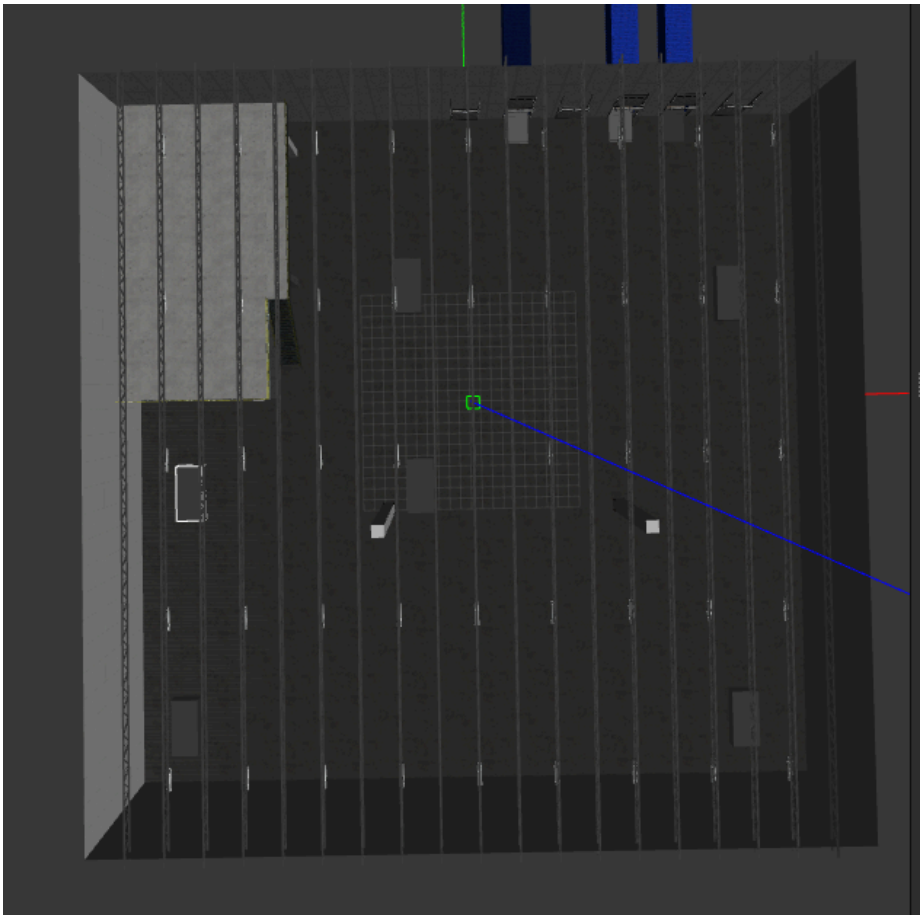
attachment:135973f5-991a-4280-ba8e-5bf08889a881:Iter0-final.webm

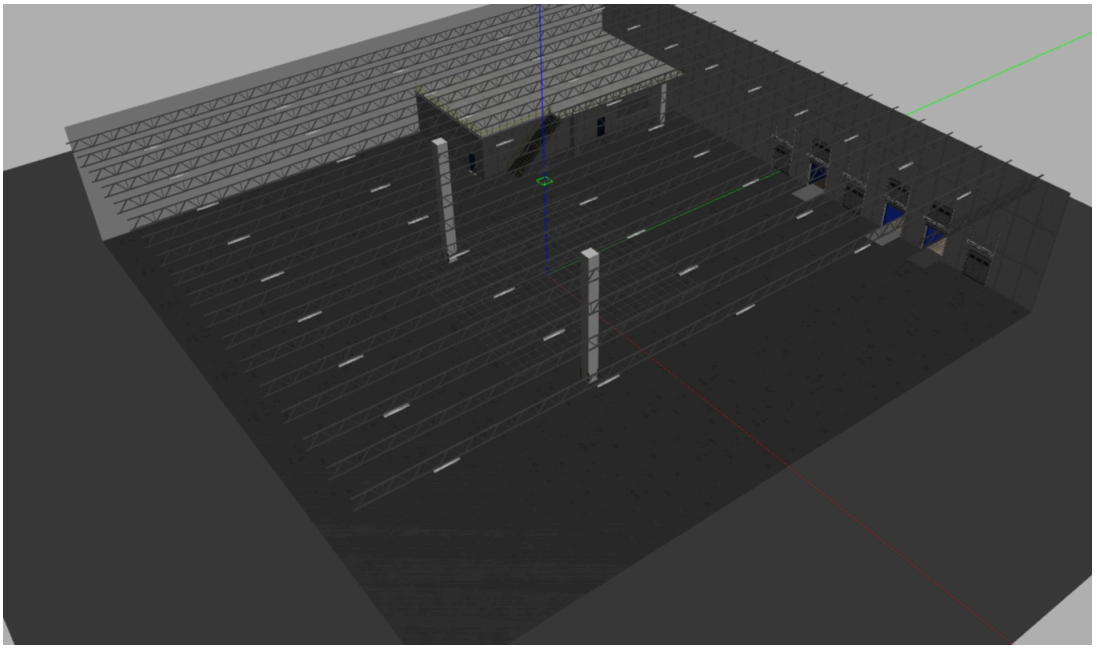
- 로봇이 Waypoint 1개까지 이동한 뒤 가짜 번호판을 인식하고, 최종 로그가 출력되는 전체 파이프라인이 동작한다.



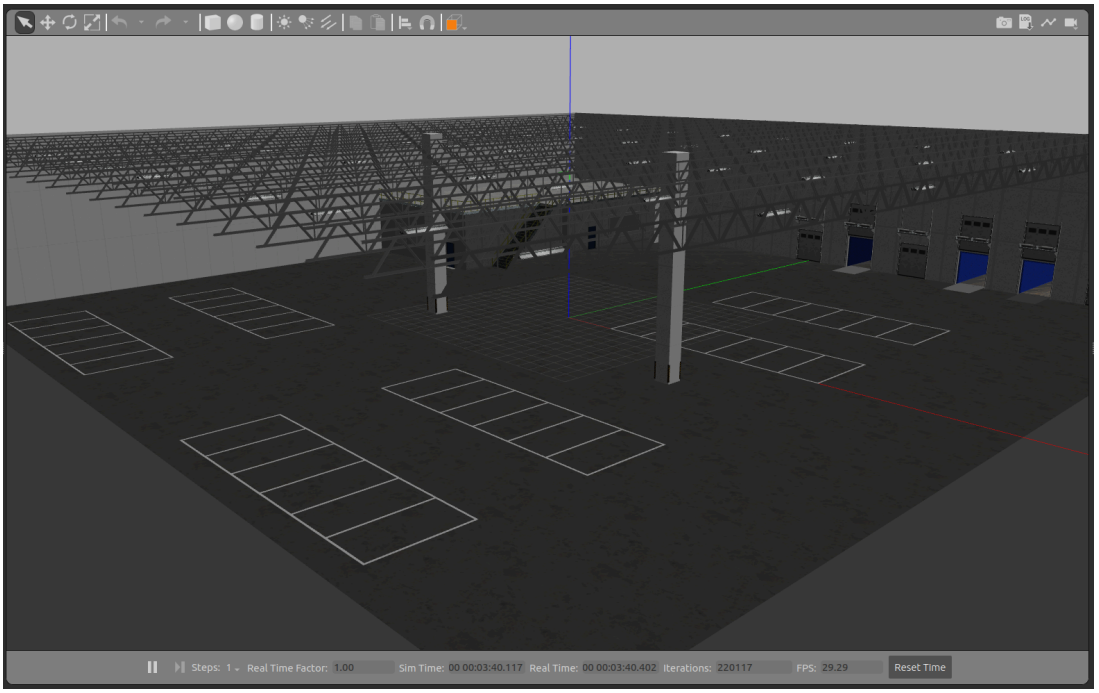
STEP1) 주차장 만들기

- 오픈소스 맵 활용
  - Distribution_Warehouse (Gazebo Fuel, by OpenRobotics)





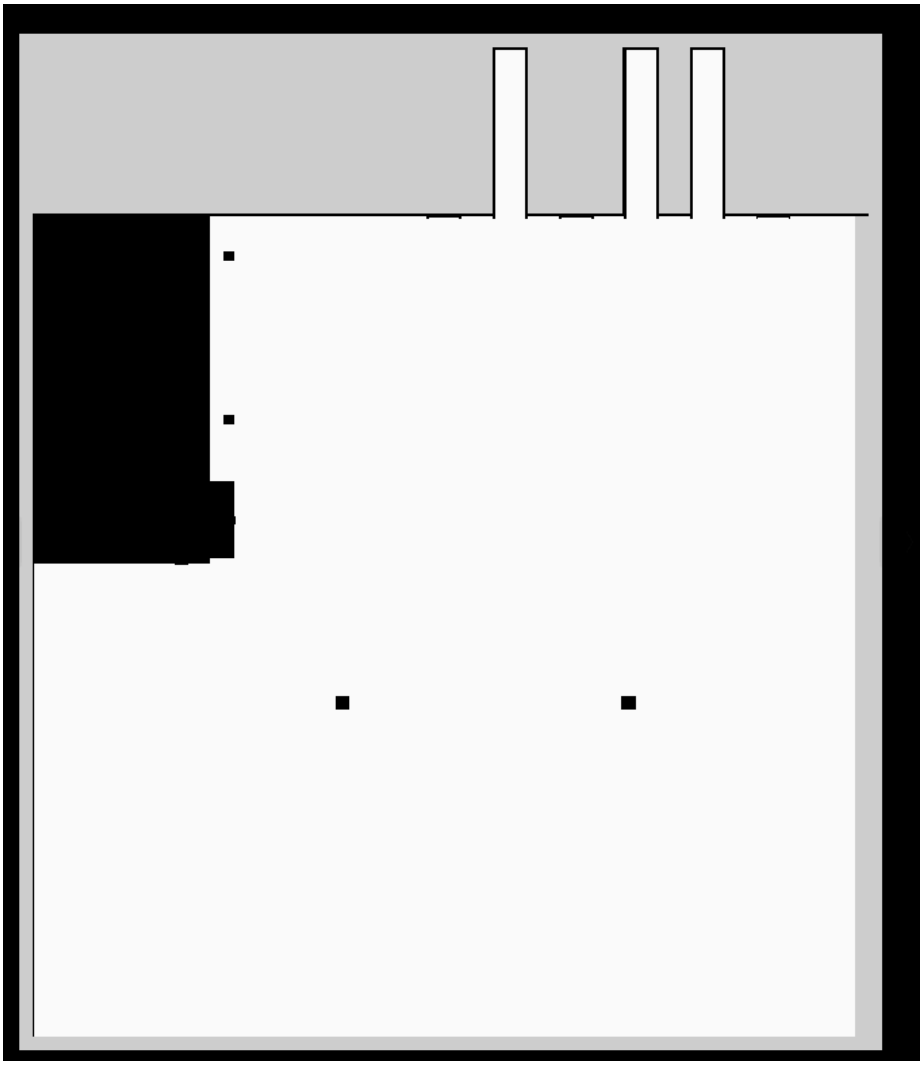
- 총 A,B,C 세구역으로 구성해 12*3=36개의 주차칸을 구성함



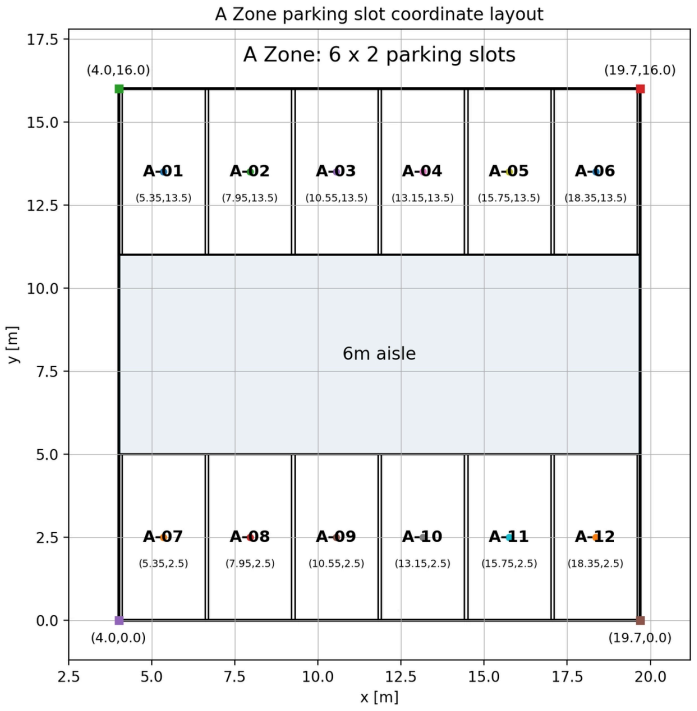

```
주차칸 폭 = 2.5m
주차칸 길이 = 5.0m
주차선 폭 = 0.1m
통로 폭 = 6.0m
```

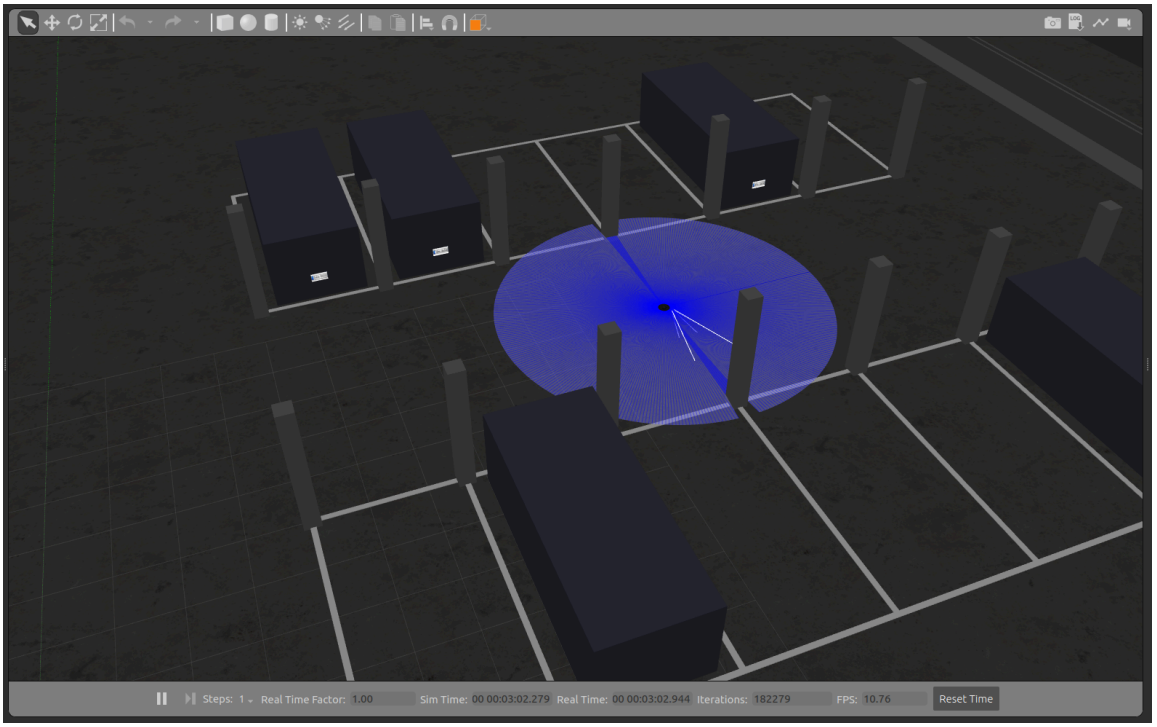
STEP2) 지도 생성 및 Localization

- 처음엔 SLAM을 시도했지만 다음 이유로 인해 포기
  1. 창고가 너무 큼 (62×73m) vs TurtleBot3 LiDAR는 최대 3.5m. → 넓은 빈 공간을 짧은 LiDAR로 teleop 주행하며 매핑하는 게 과부하.
  2. 실제로 저장한 지도의 정확도가 좋지 않음 - 특히 평평한 주차장 부분
- 대안으로 Ground-truth 기반 맵을 직접 생성했다.
  - 직접 만든 world이므로 기하정보를 알고있다.
  - map ↔ world 100% 일치



- 다만 테스트 효율성과 맵 인식률을 위해 A구역만 사용하고 기둥을 추가했다.



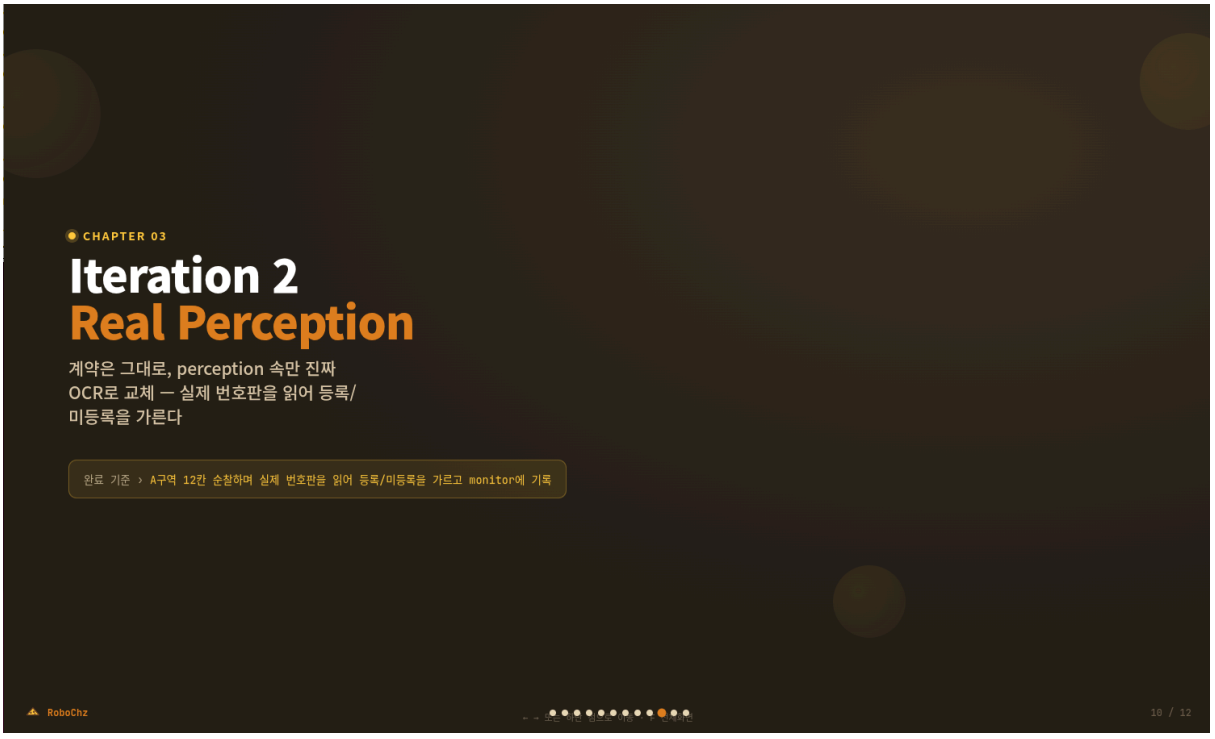


STEP3) 여러 Wapoint 순찰

- Localization (SLAM → AMCL)
  - 만든 지도를 **고정으로 로드**(map_server) + **AMCL**(파티클 필터)로 위치추정.

    - 그러나 Localization이 불안정하여 Rviz에서 로봇이동이 튕기는 현상이 지속해서 발생했다.
- 다중 waypoint 순찰
    - wp 도착 → (fake)인식 → 다음 wp → ..
    - A구역 6칸을 순서대로 한 바퀴 완주에 성공

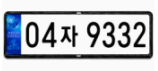
attachment:d1067893-5458-49bc-9ed5-4d515e62d6f3:Screencast_from_06-19-2026_050246_AM.webm



STEP1 차량/번호판 환경 구성

번호판 준비방법

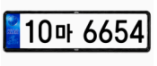
- 구글에서 적합한 번호판 하나 찾아서 ChatGPT에게 임의의 차량번호로 생성해달라고 요청 → 총 12개의 차량번호판 수집



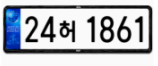
04자9332.png



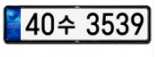
10나0303.png



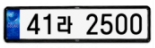
10마6654.png



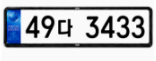
24허1861.png



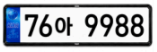
40수3539.png



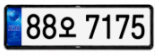
41라2500.png



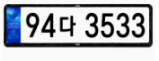
49다3433.png



76아9988.png



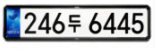
88오7175.png



94다3533.png



123가4568.png



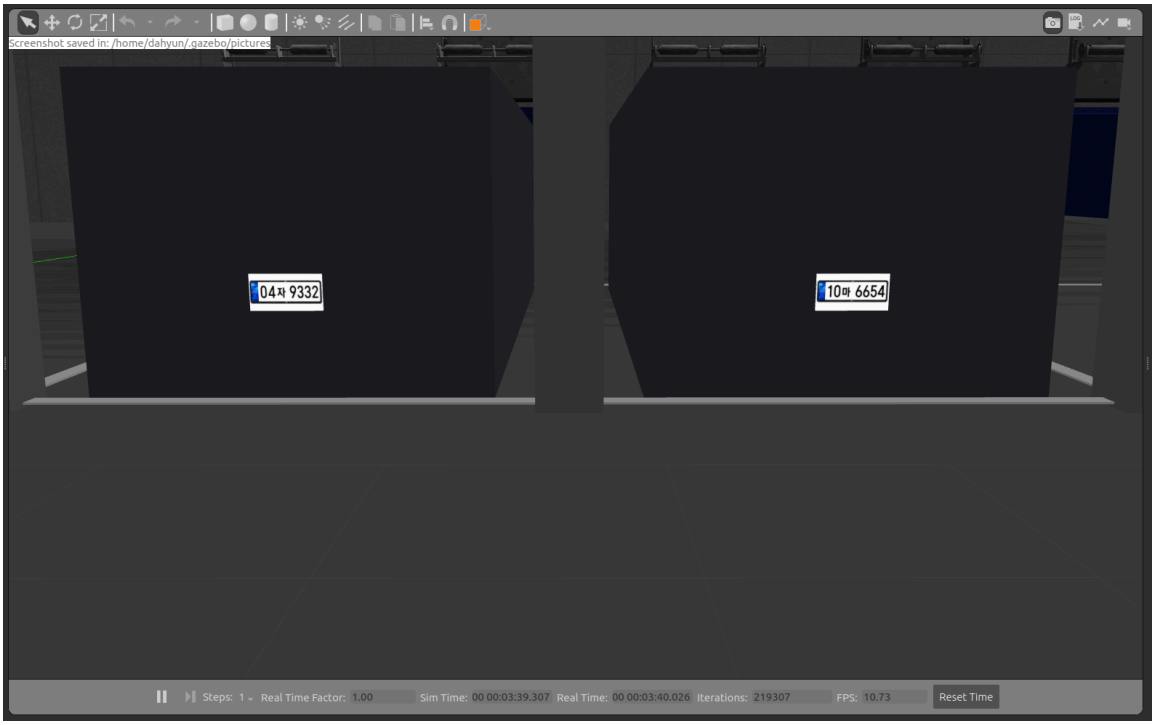
246두6445.png

- OCR 인식 테스트 결과
  - https://github.com/yunwoong7/korean_ocr_using_paddleOCR

파일(=정답)	예측	일치	평균 conf
04자9332	04자9332	✅	0.9993
10나0303	10나0303	✅	0.9428
10마6654	10마6654	✅	0.9963
123가4568	123가4568	✅	1.0000
24허1861	24허1861	✅	0.9989
40수3539	40수3539	✅	0.9996
41라2500	41라2500	✅	0.9995
49다3433	49다3433	✅	0.9996
76아9988	76아9988	✅	0.9444
88오7175	88오7175	✅	0.9999
94다3533	94다3533	✅	0.9996

완전일치 11/11 (100%) · 문자정확도 100% · 전체 평균 confidence 0.9891

- 박스차량 5대에 번호판 부착



STEP2 Robot Description 작성

- URDF 및 model.sdf 파일 생성 (카메라 90도 마운트)

  attachment:4bd2eb3a-f91c-4230-acc0-7de1d48a30d9:Iter2-camera mount.webm

- 정적장애물 추가 (콘, 배리어, 전기박스, 소화기, 선반, 정지표지판)

랜드마크	좌표 (x,y)	위치	레인거리
barrier	(4.4, 8.0)	좌측 벽 진입부 (세로)	1.5m
ebox	(10.0, 10.7)	통로 상단 가장자리(좌)	1.2m
stop_sign	(12.5, 10.7)	통로 상단 가장자리(중)	1.2m
cone	(16.5, 5.3)	통로 하단 가장자리(우)	1.2m
shelf	(19.3, 12.5)	우측 벽 상단 코너	3.0m

Step3 Rule-based , Nav Static TF

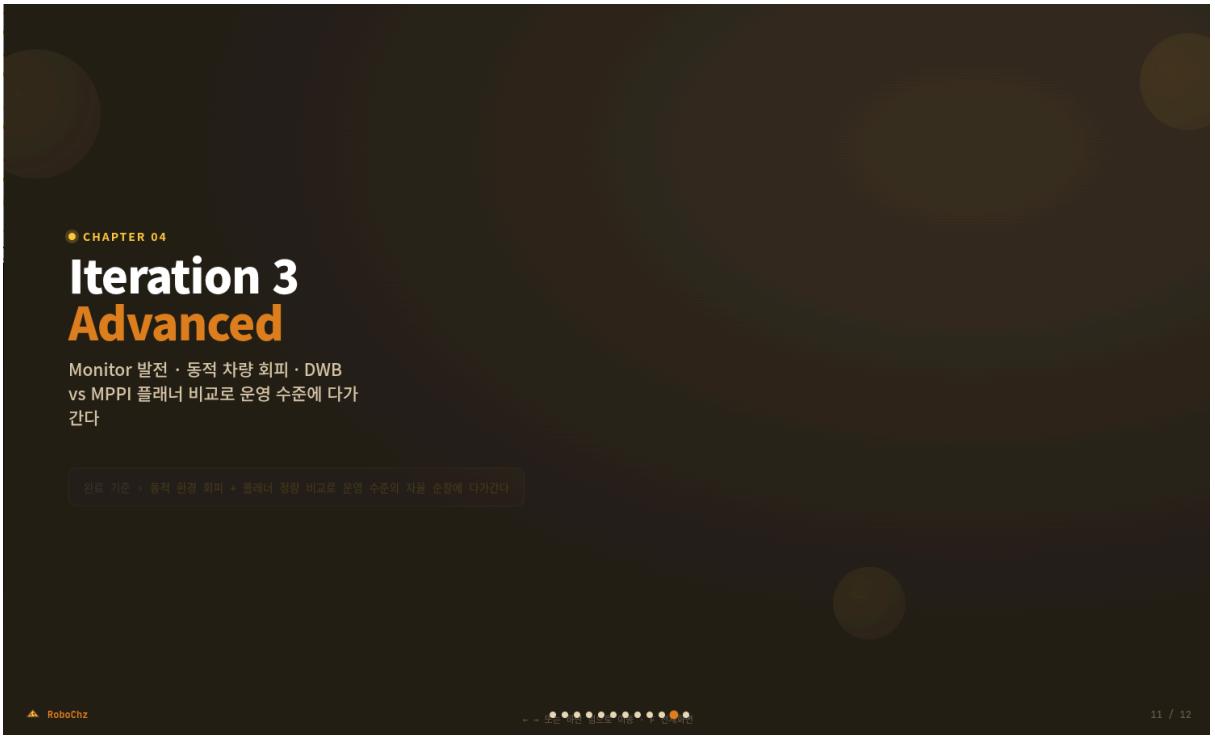
- Rule-based

- 정적장애물에 대응하지 못함 (2분대)

  attachment:1e7f250c-a39b-43d0-b1cd-7a051b464293:Iter2-Final.webm

- Nav Static TF (DWB)
  - 정적장애물에 대응함 (2분10초대)

    attachment:ec516613-61fa-435d-af14-6face9370e75:Iter2-Final.webm



Classic Planner

- DWB(Dynamic Window B): Nav2에 기본 탑재되어 있어 별도 설치 없이 바로 사용 가능하다. 속도 샘플링 방식으로 계산이 가볍고 빠른 반응이 장점이나, 경로의 부드러움이 떨어지고 좁은 공간에서 한계를 보일 수 있다.
- MPPI(Model Predictive Path Integral) : 짧은 시간 내에 수많은 경로 후보를 시뮬레이션하여 장애물을 피하면서 목표에 가장 잘 도달하는 경로를 선택한다. 이중주차처럼 예상치 못한 위치에 장애물이 생겼을 때 유연하게 우회 경로를 찾는 데 강점을 보이며, 진입 차량과 같은 동적 장애물에도 DWB보다 자연스럽게 대응한다.

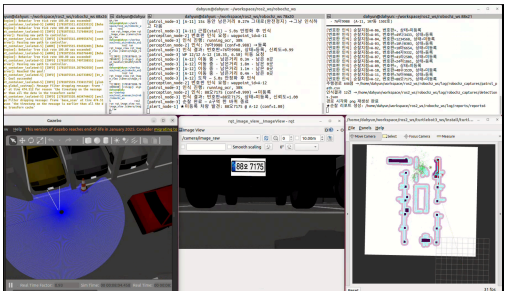
```
# 터미널 A — sim ( dwb로 돌릴땐 controller:=dwb 추가)
ros2 launch robochz_bringup sim.launch.py

# 터미널 B — app (monitor 제외)
ros2 launch robochz_bringup app.launch.py

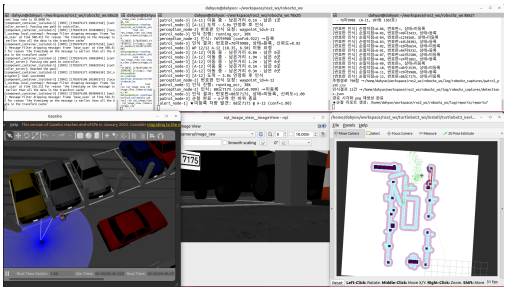
# 터미널 C — ★ monitor 별도 (시작 시 등록차량 목록 출력 + 리포트 담당)
ros2 run robochz_monitor monitor_node --ros-args -p use_sim_time:=true
```

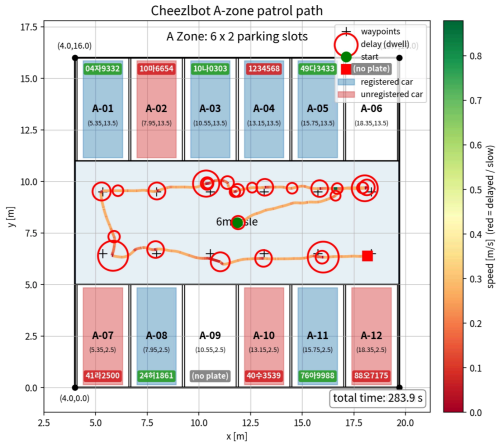
1. Mippi

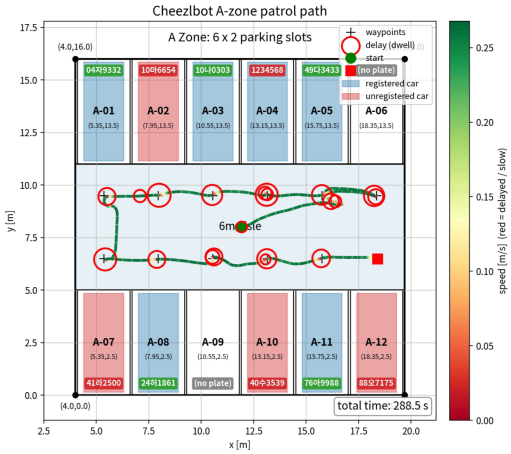
attachment:24df4bba-8706-4564-8cda-5a1674201eb5:MIPPIRealFinal.webm



2. DWB







- Mippi vs DWB 컨트롤러 비교

지표	MPPI (reports6)	DWB (reports7)	해석
완주 시간	**283.9s**	288.5s	MPPI 4.6s 빠름
경로 길이	**36.0m**	39.5m	DWB +3.5m(+10%) 더 돌아감
기록점 수	646	798	DWB가 더 많이 꿈틀(보정 기동 ↑)

1. 주행 효율 (Efficiency)

- MPPI 우세
- 주행거리: 36.0m vs 39.5m (-8.9%)
- 완주시간: 4.6초 단축
- Trajectory Points: 646 vs 798
- → 동일 목표를 더 짧고 빠르게 주행
- → 불필요한 경로 수정(Oscillation) 감소

---

2. 장애물 회피 성능 (Obstacle Avoidance)

- MPPI: 예측 기반의 부드러운 곡선 회피
- DWB: 보수적 회피 + 경로 흔들림 증가
- → 실제 실험에서 DWB는 동적차량에 밀림
- → MPPI가 더 안정적인 우회 경로 생성

---

3. 속도 특성 (Velocity Profile)

- DWB: 일정한 속도 유지
- MPPI: 속도 변화 폭이 큼
- 최대 속도는 이상치 가능성 존재
- → 절대 최고속도보다 주행 적응성(Adaptability) 에 의미

---

4. 인식 결과 일치 (Perception Consistency)

- 번호판 인식 결과 완전 동일
  - 등록 6
  - 미등록 4
  - 빈칸 2
- → Perception/OCR 성능 차이 없음
- → 결과 차이는 순수 Local Planner 성능 차이

---

5. 결론 (Takeaway)

- 정적 환경: DWB도 충분
- 동적 환경: MPPI 우세
  - 더 짧은 경로
  - 더 빠른 완주
  - 더 부드러운 회피
  - 더 안정적인 주행

THANK YOU

치즐봇은 계속 순찰 중

골격에서 시작해 진짜 순찰·인식·고도화까지 — 네 번의 Iteration으로 운영 수준의 자율 순찰 단속에 도달했다.

DONE
Iteration 0 · 1 · 2 · 3

TEAM
로보치즈 · 최다현 2277032



RoboChz

12 / 12